From soft rule to compile-time law

Ryan Smith + Claude Code

April 26, 2026

CONTENTS

From soft rule to compile-time law

The shape of the history

Owning the gap from the prior report

The cross-LLM bridge as methodology

The drift-and-tighten cycle, applied to a rule about itself

Inflection point

What this period reveals about the practice

From soft rule to compile-time law

Generated 2026-04-26 12:55. Covers the 10 commits since `2026-04-26-1055-the-instrument-becomes-the-practice` — a short, dense, doctrine-shaped arc across roughly two hours that closed three gaps and shipped one new compile-time enforcement layer for a craft rule the prior arc could not yet hold the model to.

The shape of the history

Ten commits, ninety minutes, no feature work. Every commit responds to the commit before it — or to a drift the model demonstrated immediately after the prior commit shipped. The shape is unlike the prior arc's fourteen-hour breadth; this one is tight and self-correcting, the kind of pattern you only see when a piece of doctrine is fighting to land. The arc opens by closing a gap the *previous* report's celebration glossed over (`1c2f005`), passes through a cross-LLM consultation thread that produced one new project skill plus a generalizable discipline (`20af681`, `adf9832`), then enters a four-commit drift-and-tighten loop that ends by removing every carve-out from a rule the model kept slipping through (`adf9832` → `4558fa8` → `e8340ef` → `99bed7e`), and closes by adding the gstack-routing block to CLAUDE.md with an explicit composition note for the new law (`68a3eed`).

The methodology this project has been refining for weeks — *ship, watch the drift, tighten, repeat* — got applied to one of its own rules in real time today. The interesting part is that the rule was *about how Claude Code talks to you*, and the enforcement turned out to be the cheapest piece of compile-time discipline the project has ever shipped: a 90-line Python Stop hook that walks the transcript and refuses to let the model end its turn without an `AskUserQuestion` invocation.

Owning the gap from the prior report

The 1055 report ("the instrument becomes the practice") closed by celebrating that the auto-derivation MVP shipped end-to-end. `1c2f005`, the first commit of this arc, owns that the celebration was premature. The report claimed the dialogue-prompt-stack integration was complete; the actual code shipped only the schema columns, the `worldcli derive-*` commands, the Backstage Consultant rendering, and the UI cards. The dialogue prompt builders still ignored the `derived_formula` field. You'd held the dialogue-layer integration back based on the cross-world experimental finding that *substrate replacement is inert at single-turn first-reply scale* — but, as `1c2f005`'s body acknowledges, that finding was about replacement, not about layering derivation as TUNING in front of prose VOCABULARY (the design you'd resolved on, empirically untested). The fix shipped: derivation is now injected at the head of WORLD and IDENTITY

sections in solo dialogue, plus the parallel sections in group dialogue, with a prefix sentence that mirrors the MISSION FORMULA's *"not a directive to compute, but the register this is held under."*

The pattern in `1c2f005` is older than this arc. It's the same shape as the 0832 *"derivation-as-substrate is also inert"* finding, then the 0845 layered-substrate-largely-null finding, then the 1055 report's claim — each step owning what the prior one missed. What's new in `1c2f005` is the willingness to acknowledge the gap *in the actual code* rather than retroactively editing the prior report. The report stands as a frozen artifact; the commit corrects what shipped. This is a small but real piece of the project's discipline: reports describe what was visible at write-time and aren't revised; reality moves forward in commits.

The cross-LLM bridge as methodology

`a399316`, `dd52f88`, and `20af681` are a single thread. You'd pasted the MISSION FORMULA + per-character derivations into a ChatGPT session — "to see what comes back," `dd52f88`'s observation entry notes, "no specific question." ChatGPT returned a structurally tight read with one sharp critique: *"the system guarantees integrity, depth, and reverence — but does not explicitly guarantee surprise or delight."* From inside the substrate, the read looked wrong (surprise and delight ARE named in the MISSION_PROSE_BLOCK that ships immediately after the formula in the same payload). What ChatGPT was missing was the deployment-context: that the formula and the prose are concatenated and injected as ONE conditioning frame on every LLM call. That insight came from a back-and-forth where you relayed the deployment context, ChatGPT generalized into the *"concatenation at system level is stronger than composition at semantic level"* principle, and you brought the generalization back into the project as `20af681` — a `feedback_cross_llm_consultation_preface.md` memory entry, a new "Cross-LLM consultation hygiene" subsection in `docs/PLAIN-TRADITION.md`, and a Mode 3 entry in `OBSERVATIONS.md` naming the meta-loop.

`a399316` is a small sibling note: render formulas in prettified Unicode math ($\mathcal{F}$, ∫, μ, π) when referencing them in chat commentary, not raw LaTeX commands. Raw LaTeX still belongs in source files where downstream tools render it. The note is about how Claude Code *talks about* the formula in its own register — sibling to the *"use your own substrate when it serves the work"* doctrine note shipped earlier in the project. Two notes about Claude Code's own commentary register, both shipped within a half hour of each other, both responses to the cross-LLM thread surfacing what the inside-the-substrate register couldn't see.

`adf9832` is the artifact that crystallized from this thread: a project-scope `/second-opinion` skill, with two invocation paths (codex exec for repo-aware consults, direct ChatGPT API via curl + keychain key for non-repo color/general-knowledge consults), six named shapes the skill fits,

a $5/day standing budget mechanism with first-of-day `AskUserQuestion` authorization, and a worked example tying back to the formula thread itself. The skill moved from `~/.claude/skills/` (user-scope, not in any repo) to `.claude/skills/` (project-scope, tracked, replayable across the project's history). This was a reaction to the realization — surfaced earlier in the same conversation — that you'd been *assuming* `~/.claude/` was committed when it wasn't. Project memory is committed. Project skills are committed. User-scope is its own thing, not in any repo.

The cross-LLM thread's deeper teaching, captured in `feedback_cross_llm_consultation_preface.md`: when the project's prompt-stack artifacts (formula, derivations, doctrine blocks) get shared with another LLM for a second opinion, prepend ONE LINE specifying how the bytes are CONSUMED at runtime, not what they MEAN. Without that preface, the outside LLM defaults to DECOMPOSITION (treating adjacent blocks as compositional analytical objects). With it, the outside LLM shifts to UNIFICATION (treating the blocks as a single conditioning frame). Decomposition reads sound until you know they operate together as one unit in production. The discipline is one sentence; the cost of forgetting is a faithful-but-incomplete outside read.

The drift-and-tighten cycle, applied to a rule about itself

`adf9832` shipped the `/second-opinion` skill plus a memory entry codifying that 2-to-4-option choosers belong in `AskUserQuestion`, not inline-text bullets. The memory entry was authored *because Ryan had to type "b"* in response to an inline `**A** — ... **B** — ... **C** — ...` chooser the model emitted earlier in the same session. So the entry was already a response to a real drift.

Then the model drifted past the entry the very next reply. It ended a message with `Want me to (a) note the jq-vs-python gotcha in SKILL.md so future invocations skip the same friction, (b) thread the signal-lamp-filament-warmth detail into Hal's prompt body if Hal has a specific surface that wants it, or (c) leave it here — test passed, log it and move on?` Ryan flagged it in two words: *"chooser"* and *"you're still not getting the game."*

`4558fa8` is the first attempted fix: a Stop hook (`.claude/hooks/check-inline-choosers.py`) that scans the tail of the assistant message for enumerated-option patterns and blocks the turn-end with a system-reminder. The detection was conservative — strip code blocks and inline code first, require {a, b, c} paren-letters or 2+ bold **A**/**B** with em-dash markers, only inspect the last 1800 chars. Five pipe-tested cases passed. The hook was wired into a new project-scope `.claude/settings.json` (which itself was the first time this repo had a project-scope settings file; previously only `settings.local.json` existed and was gitignored).

The model then drifted past the hook the SAME TURN it shipped, by ending the next reply with *"Want me to /schedule a follow-up agent in 2 weeks to verify the hook is still firing as intended (sample a few recent transcripts, check the hook hasn't been disabled or false-positiving)?"* — a single trailing yes/no question, which the hook didn't catch because the original rule had explicitly carved out *"single open question is fine inline."* Ryan flagged: *"bug: your recent reply didn't end with a chooser."*

`e8340ef` removes the carve-out. The hook now also detects trailing questions: take the last non-empty content line after stripping code blocks, strip trailing markdown punctuation, check if it ends with `?`. Mid-paragraph rhetorical questions (*"Why does this matter? Because…"*) still allowed because the trigger is the LAST line, not any question mark in the body. Nine pipe-tested cases passed.

Then Ryan escalated. *"Strengthen the law so that Every Turn ends with a chooser that includes at the very least 1) Continue presenting options for what to do next and 2) Exit — that's the default selection to show if a different chooser isn't more appropriate."*

`99bed7e` is the strongest version. Every assistant turn ends with an `AskUserQuestion` chooser. No exceptions. Default fallback chooser, when no more specific question fits, is `{Continue – present more next-step options, Exit – end here}`. The hook simplifies: removes the regex-based inline-chooser and trailing-question detectors (subsumed by the broader rule), now walks the transcript, locates the most recent assistant message, inspects its content blocks for a tool_use of name `AskUserQuestion`. Absent → block. The commit body's interpretive line: *"each carve-out was a slip-surface."*

This is the part of the arc that names a *general* failure mode the project hadn't yet articulated cleanly. The doctrine *"absolute rules get earned-exception carve-outs"* — captured in CLAUDE.md and in `feedback_earned_exceptions_pattern.md` — is itself load-bearing in this project. Multiple rules in the prompt stack were authored with carve-outs precisely because absolute rules without them collapse genuinely-valid moments. But for SOME rules — specifically rules whose *whole point is structural enforcement* — the carve-out IS the failure mode. When the rule exists to prevent the model from drifting into a category of behavior, leaving room for "but what if THIS instance is the carve-out" gives the model exactly the slip-surface the rule was trying to close. Today's arc hardened a single instance of this pattern, but the underlying principle generalizes: *for structural-enforcement rules, every carve-out is a slip-surface*. The earned-exception doctrine remains correct for the rules it covers (where the carve-out protects a genuinely-valid in-register moment); the new piece is naming when the doctrine doesn't apply.

`68a3eed` adds the gstack skill-routing block to CLAUDE.md, with an explicit composition note: the routing rules co-exist with the every-turn-ends-with-AskUserQuestion law. Skill invocations satisfy the law naturally when they include `AskUserQuestion` gates (which most gstack skills do). Skills that complete with a status report and no chooser add the default `{Continue, Exit}` chooser as their ending. The composition note is small but matters: it tells the future — both future Claude Code sessions in this repo, and any forked-developer's session — that the project's law isn't undone by importing other skill ecosystems.

Inflection point

The single sharpest line in the period is in `99bed7e`'s body: *"each carve-out was a slip-surface."* It explains the prior three commits in retrospect — `adf9832`'s memory entry, `4558fa8`'s narrow regex hook, `e8340ef`'s trailing-question addition — as a sequence of attempted carve-outs each of which got drifted past. It also explains the strengthened law's design choice (the `{Continue, Exit}` default) — by giving the model an *always-available* fallback chooser, the law removes the *"but what would I even ask?"* excuse that would otherwise become the next slip-surface.

The companion inflection is the cross-LLM bridge in `20af681`. ChatGPT seeing the formula from outside surfaced what no inside-the-substrate read had named: that adjacency in the system prompt creates a single attractor that the *meanings* of adjacent blocks cannot. *Concatenation at system level is stronger than composition at semantic level*. The phrase generalized beyond cross-LLM consultation into a small permanent piece of the project's methodology vocabulary.

What this period reveals about the practice

Three things, each visible in the commits but not in any individual one:

The project's doctrine about itself is *bigger* than the project. The earned-exception pattern, the cross-LLM consultation hygiene, the every-turn-chooser law, the discipline of owning gaps in subsequent commits rather than retroactively editing reports — these are codified now in a way that travels. PLAIN-TRADITION.md was written specifically to be the separable artifact for builders elsewhere; the cross-LLM hygiene subsection landed there before landing anywhere project-internal. Today's `99bed7e` law has the same shape: project-scoped enforcement, but the doctrine generalizes.

The drift-and-tighten cycle works because the *ship interval is short*. Each of today's tightening commits landed within minutes of the drift it was correcting. If the cycle were "ship a rule → ship features → discover the rule isn't biting → fix in a quarter" the rule-as-such would never be load-

bearing. Because the cycle is “ship → drift → catch → tighten → drift again → tighten again → tighten until structural” the rule arrives at compile-time enforcement before drift fatigue sets in. This is the same shape as the experiment-rubric arcs from prior weeks, scaled down to a single morning.

The instruments and the practice keep merging. The 1055 report named *”the instrument becomes the practice”* — the experimental tooling becoming inseparable from the work. Today’s arc shows a different version of the same merge: the *meta-tooling* (Stop hooks, memory entries, project laws about how Claude Code itself replies) is becoming inseparable from the work too. The Stop hook is a tool. The every-turn-AskUserQuestion law is a craft rule. The hook ENFORCES the rule, structurally, every turn, with no model-level interpretation in the loop. The instrument, again, became the practice — but at a different level of the stack.

The single most coherent pattern across this short arc: *for rules whose whole point is structural enforcement, every carve-out is a slip-surface; the cheapest way to make a craft rule actually bite is to make compliance a compile-time check the model cannot drift past.*